CEUB

EDUCAÇÃO SUPERIOR

Desenvolvimento de Sistemas

ceub.br

CEUB
EDUCAÇÃO SUPERIOR

Aula 04 –

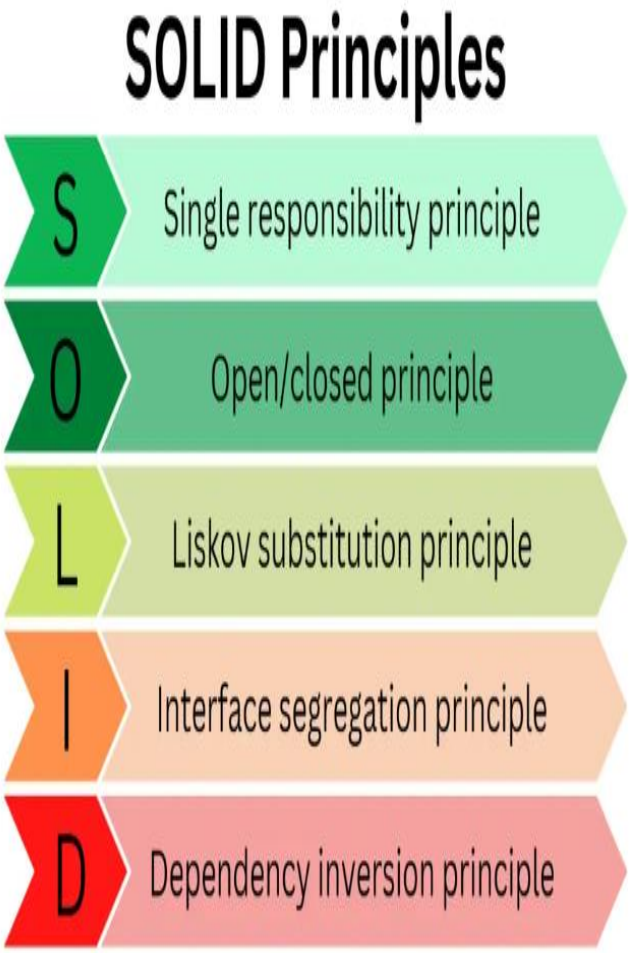
Boas práticas de desenvolvimento back-end com C#

Agenda

- SOLID
- Princípio - Single Responsibility Principle
- Open-Closed Principle (Princípio Aberto-Fechado)
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

SOLID

SOLID



O SOLID é um acrônimo que representa conjunto de princípios de design de software, proposto por Robert C. Martin nos anos 2000, que podem ser usados para criar código mais coeso, flexível, reutilizável e fácil de manter.

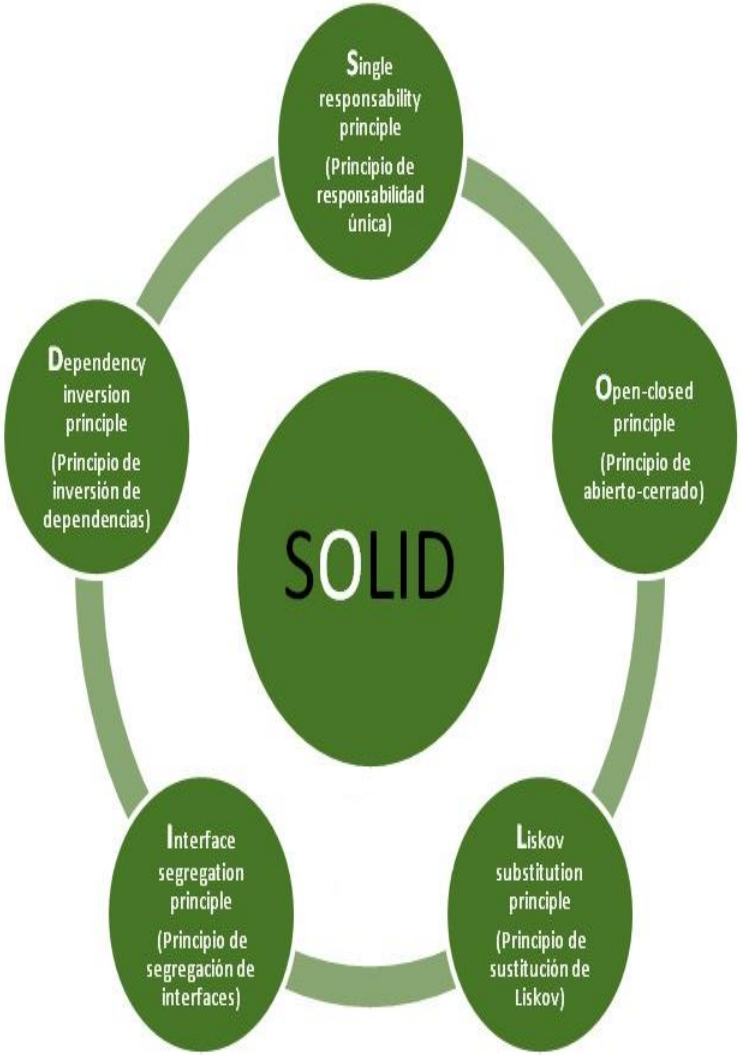
Ao aplicar os princípios SOLID à orientação a objetos, você pode criar código mais eficiente e de alta qualidade.

SOLID - Princípios

Os princípios do SOLID devem ser aplicados para se obter os benefícios da orientação a objetos, tais como:

- **Possibilitar a facilidade em se manter, adaptar e ajustas alterações de escopo de um projeto OO;**
- **Permitir que o código criado possa ser testável e de fácil entendimento**
- **Seja extensível para alterações com esforço reduzido**
- **Implemente o reuso na prática**
- **Permita que uma aplicação seja utilizada o máximo possível, o código tem que ficar pelo tempo que a aplicação seja necessário**

SOLID - Princípios



Ao utilizar os princípios do SOLID é possível evitar os seguintes problemas:

- **Dificuldade de isolar funcionalidades**
- **Duplicação de código, ou seja, não se consegue alterar um código sem que a mudança deva ser realizada em várias partes do projeto;**
- **Código sem estrutura ou padrão (Código macarrônico);**
- **Dificuldade na testabilidade**

SOLID
Princípio - Single Responsibility Principle

O Princípio da Responsabilidade Única é aquele que afirma que uma classe deve ter apenas uma razão para mudar, ou seja, ela deve ter uma única responsabilidade.

Uma classe deve representar uma única funcionalidade ou conceito do sistema.

Isso promove o baixo acoplamento e alta coesão, facilitando a manutenção, teste e reutilização do código.

SOLID

S Single responsibility principle

Exemplo

Situação Problema:

A classe Cliente atualmente tem a responsabilidade de gerenciar os dados do cliente e enviar e-mails de notificação quando necessário.

Além disso, ela também pode gerar faturas para enviar aos clientes cadastrados.

Isso viola o SRP, pois a classe tem mais de uma razão para mudar.

```
public class Cliente
{
    public string Nome { get; set; }
    public string Endereco { get; set; }
    public string Telefone { get; set; }

    public Cliente(string nome, string endereco, string telefone)
    {
        Nome = nome;
        Endereco = endereco;
        Telefone = telefone;
    }

    public bool Validar()
    {
        // Valida o nome, endereço e telefone do cliente.
        return true;
    }

    public string GerarFatura()
    {
        string fatura=string.Empty;
        // Cria uma fatura para o cliente.
        return fatura;
    }
    public string SalvarCliente()
    {
        // Lógica para salvar o cliente no banco de dados
      return ("Cliente salvo no banco de dados.");
    }

    public void EnviarEmail(string nome, string email)
    {
        // Envia um email para o cliente com a fatura anexada.
    }
}
```

SOLID

S Single responsibility principle — **Exemplo**

Solução do Problema:

SOLID

S Single responsibility principle — **Exercícios**

Sistema de Gestão de Funcionários
Imagine que você está desenvolvendo um sistema de gestão de funcionários para uma empresa. O sistema precisa realizar as seguintes operações:

Adicionar um novo funcionário ao sistema.
Calcular o salário de um funcionário com base em seu cargo e horas trabalhadas.
Gerar relatórios de desempenho dos funcionários.
Para praticar o SRP, seu objetivo é refatorar o código para garantir que cada classe tenha apenas uma única responsabilidade.:

SOLID
Princípio - Open-Closed Principle (Princípio Aberto-Fechado)

O Princípio Aberto-Fechado é aquele que diz que As classes devem ser abertas para extensão, mas fechadas para modificação.

Isso significa que você deve ser capaz de adicionar novas funcionalidades a uma classe sem precisar modificar o código-fonte original.

Promove o uso de abstrações, interfaces e padrões de projeto como Strategy e Factory Method.

SOLID

O Open/closed principle — Exemplo

Imagine que você está desenvolvendo um sistema de pagamento que precisa calcular o valor total a ser pago por uma compra, levando em consideração diferentes tipos de descontos.
Inicialmente, você implementou uma classe chamada CalculadoraDePagamento que possui um método CalcularTotal para calcular o valor total da compra.

No entanto, a implementação atual da classe viola o OCP, pois não é facilmente extensível para adicionar novos tipos de desconto no futuro sem modificar o código existente.

```
public class CalculadoraDePagamento
{
    public decimal CalcularTotal(decimal valorCompra, string tipoDesconto)
    {
        decimal valorDesconto = 0;

        // Aplicar desconto com base no tipo
        switch (tipoDesconto)
        {
            case "BLACK_FRIDAY":
                valorDesconto = valorCompra * 0.2m;
                break;
            case "CUPOM_DESCONTO":
                valorDesconto = 50;
                break;

            default:
                break;
        }

        decimal total = valorCompra - valorDesconto;
        return total;
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        // Exemplo de utilização da classe CalculadoraDePagamento
        CalculadoraDePagamento calculadora = new CalculadoraDePagamento();
        decimal totalCompra = calculadora.CalcularTotal(100, "BLACK_FRIDAY");
        Console.WriteLine($"Total a pagar: {totalCompra}");
    }
}
```

SOLID

Solução do Problema:

SOLID

Sistema de Cálculo de Desconto

Você está desenvolvendo um sistema simples de cálculo de desconto para uma loja online.

O sistema deve ser capaz de aplicar descontos de diferentes tipos, como desconto de cupom, desconto por quantidade e desconto sazonal.

Seu objetivo é projetar o sistema de forma que seja fácil adicionar novos tipos de desconto no futuro sem modificar o código existente.

SOLID

Princípio - Liskov Substitution Principle (Princípio da Substituição de Liskov)

O Princípio da Substituição de Liskov (LSP) garante que as subclasses possam substituir suas classes base sem causar problemas no programa.

Em outras palavras, se um objeto pertence a uma classe base, ele deve poder ser substituído por um objeto de uma classe derivada sem comprometer a integridade do programa.

Isso assegura que a herança seja usada de maneira consistente, sem impactar a correção do código.

SOLID

L Liskov substitution principle

Exemplo

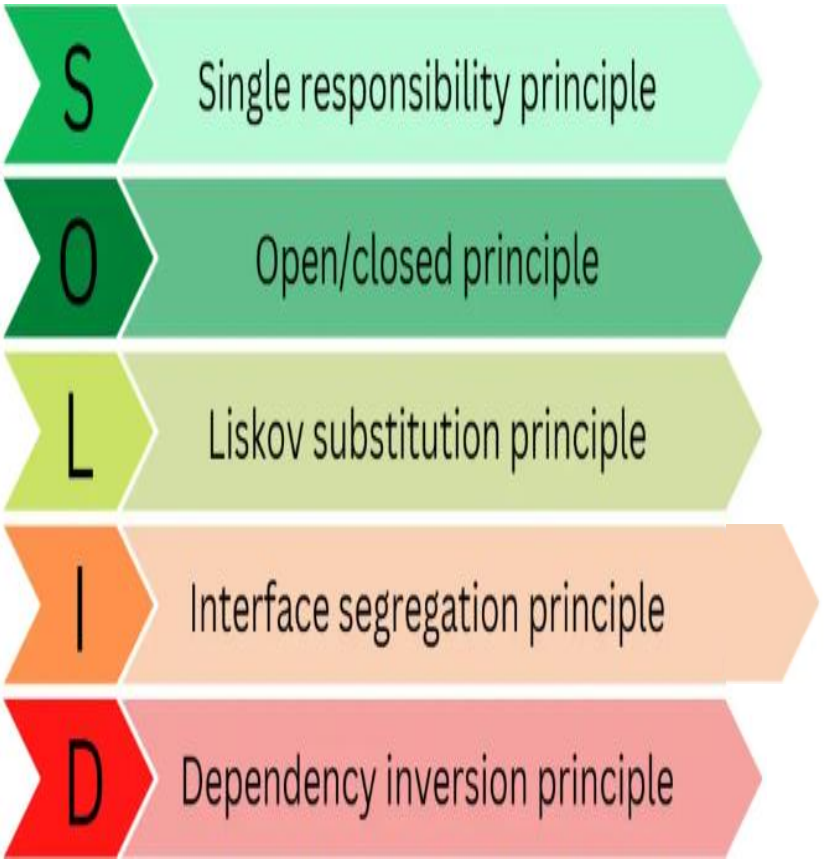
```
public class Retangulo
{
    public virtual double Altura { get; set; }
    public virtual double Largura { get; set; }
    public double Area { get { return Altura * Largura; } }
}

public class Quadrado : Retangulo
{
    public override double Altura
    {
        set { base.Altura = base.Largura = value; }
    }

    public override double Largura
    {
        set { base.Altura = base.Largura = value; }
    }
}
```

```
public class CalculoArea
{
    private static void ObterAreaRetangulo(Retangulo ret)
    {
        Console.Clear();
        Console.WriteLine("Calculo da área do Retangulo");
        Console.WriteLine();
        Console.WriteLine(ret.Altura + " * " + ret.Largura);
        Console.WriteLine();
        Console.WriteLine(ret.Area);
        Console.ReadKey();
    }

    public static void Calcular()
    {
        var quad = new Quadrado()
        {
            Altura = 10,
            Largura = 5
        };

        ObterAreaRetangulo(quad);
    }
}
```

SOLID

SOLID

Princípio - Interface Segregation Principle (Princípio da Segregação de Interfaces)



O Princípio da Segregação de Interfaces é aquele que afirma que uma classe não deve ser forçada a depender de métodos que ela não utiliza.

Ele promove a divisão de interfaces em interfaces menores e mais específicas, adequadas às necessidades das classes que as implementam.

Evita interfaces "gordas" e reduz o acoplamento entre classes.

SOLID

I Interface segregation principle

Exemplo

```
public interface ICadastro
{
    void ValidarDados();
    void SalvarBanco();
    void EnviarEmail();
}
```

```
public class CadastroCliente : ICadastro
{
    public void ValidarDados()
    {
        // Validar CPF, Email
    }

    public void SalvarBanco()
    {
        // Insert na tabela Cliente
    }

    public void EnviarEmail()
    {
        // Enviar e-mail para o cliente
    }
}
```

```
public class CadastroProduto : ICadastro
{
    public void ValidarDados()
    {
        // Validar valor
    }

    public void SalvarBanco()
    {
        // Insert tabela Produto
    }

    public void EnviarEmail()
    {
        // Produto não tem e-mail, viola o principio ISP Interface Segregation Principle
        throw new NotImplementedException("Esse metodo não serve pra nada");
    }
}
```

SOLID

Princípio - Dependency Inversion Principle (Princípio da Inversão de Dependência)

Princípio da Inversão de Dependência afirma que os módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações.

Ou seja, as classes devem depender de abstrações, não de implementações concretas.

Isso significa que as classes não devem depender diretamente de outras classes, mas sim de interfaces.

Promove o uso de interfaces e injeção de dependência para desacoplar componentes e facilitar a substituição de implementações.

SOLID

D Dependency inversion principle — **Exemplo**

Imagine um sistema onde a classe Aluno depende diretamente da classe Turma para obter informações sobre a turma em que está matriculado.

```
public class Aluno
{
    private Turma _turma;

    public Aluno(Turma turma)
    {
        _turma = turma;
    }

    public string ObterNomeTurma()
    {
        return _turma.Nome;
    }
}

public class Turma
{
    public string Nome { get; set; }
}
```

```
public class Program
{
    public static void Main(string[] args)
    {
        var turma = new Turma { Nome = "Turma A" };
        var aluno = new Aluno(turma);

        var nomeTurma = aluno.ObterNomeTurma();

        Console.WriteLine($"Nome da turma: {nomeTurma}");
    }
}
```

A classe Aluno depende da classe Turma para ser instanciada. Se a classe Turma for modificada, a classe Aluno também pode precisar ser modificada.
Essa dependência direta torna o código menos flexível e rígido.

SOLID

D Dependency inversion principle

Solução

Referências:

https://owasp.org/www-community/attacks/

https://www.dio.me/articles/a-evolucao-da-arquitetura-de-software-backend

https://4future.com.br/index.php/2023/12/04/compreendendo-os-principios-solid/

https://medium.com/beelabacademy/princ%C3%ADpios-de-s-o-l-i-d-em-c-guia-pr%C3%A1tico-cbb1e6584284

PIRES, E. SOLID: Teoria e Prática.
ANICHE, M. Orientação a Objetos e SOLID para ninjas. São Paulo – Casa do Código

Obrigado